

App Dev Project Report

1. Student Details

- **Name:** Debankush Ghosh
- **Roll Number:** 24F3001469
- **Email:** 24f3001469@ds.study.iitm.ac.in
- **About Me:** I am a student at IIT Madras BS Degree program with a keen interest in full-stack web development and database management. I enjoy building practical applications that solve real-world administrative problems through secure and efficient code.

2. Project Details

- **Project Title:** Trekking Management Application
- **Problem Statement:**

Adventure organizations require efficient systems to manage trekking activities involving trek organizers, staff, and participants. Currently, many trekking groups rely on spreadsheets, phone calls, or manual coordination, which makes it difficult to manage trek approvals, track bookings, avoid overbooking, and maintain trek history. The goal is to build a web application that allows an Admin, Trek Staff, and Users (Trekking) to interact securely based on roles, executing all operations strictly on the server side without JavaScript.
- **Approach:**

The application was built as a lightweight, single-file Flask architecture to maintain simplicity and fast local execution. Role-Based Access Control (RBAC) is enforced entirely through server-side HTTP session tracking. Forms utilize pure HTML **GET** and **POST** methods to update data. Critical application logic—such as real-time tracking of open slot metrics—prevents overbooking systematically on the backend before running transactional database commits.

3. AI/LLM Declaration

- I used Gemini to assist in drafting clean Vanilla CSS styling rules, structuring relational SQLite database schemas, and formatting template layout inheritance.
- The extent of AI/LLM usage is around 15%, restricted entirely to structural code formatting suggestions and documentation mapping.
- All core application architecture, state transitions, slot control validation blocks, and final testing integrations were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework and routing engine
SQLAlchemy	Object Relational Mapper to programmatically generate and interact with the database
Jinja2	Server-side template engine for rendering dynamic HTML pages containing role-specific logic
Vanilla CSS	Custom styling embedded directly in views for page layouts without using Bootstrap or external frameworks
Werkzeug	Provides secure password verification hashing structures (pbkdf2:sha256)
SQLite	Lightweight, file-based relational database used for local persistence

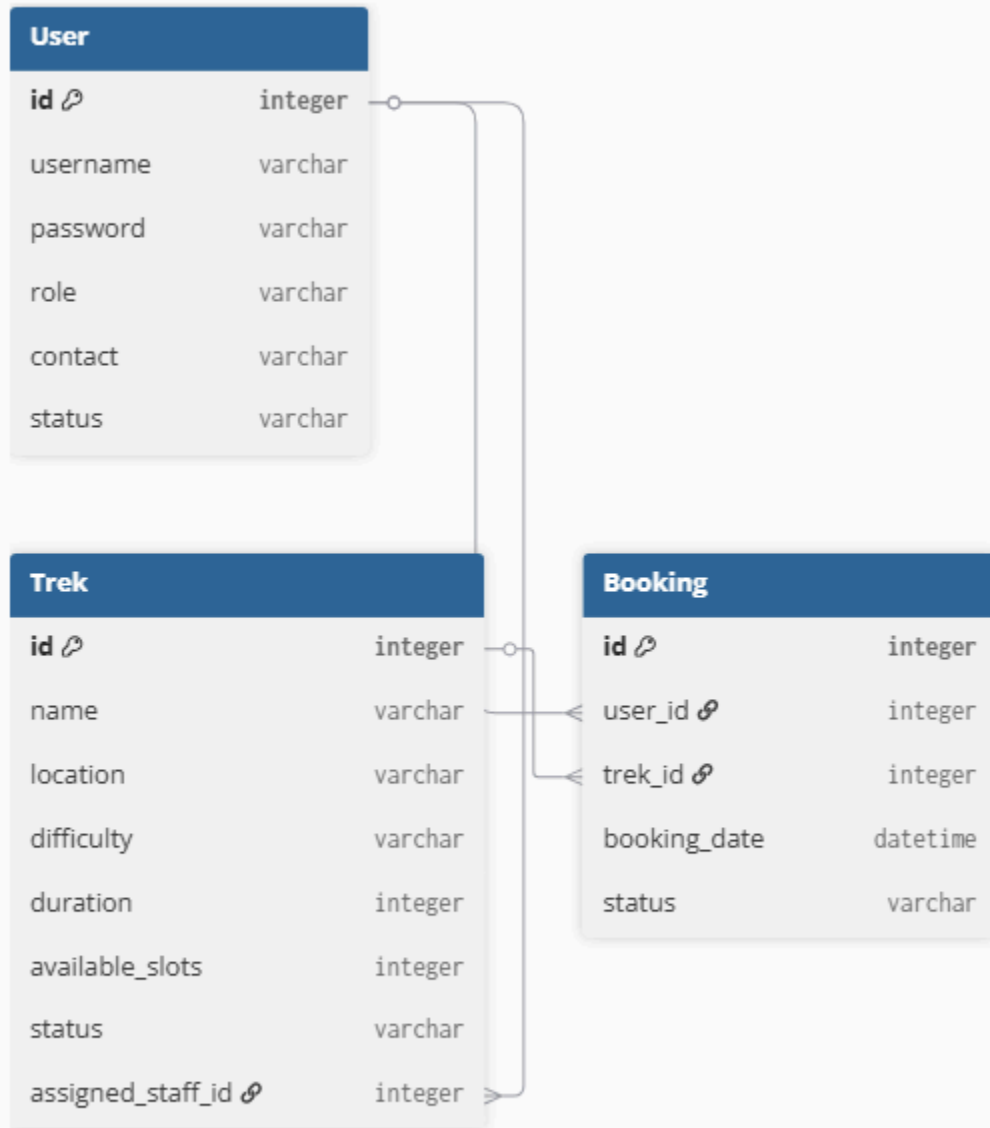
5. Database Schema / ER Diagram

Tables:

- **User** — Stores profiles for all roles (id, username, password, role, contact, status)
- **Trek** — Stores trekking event tracks created by the admin (id, name, location, difficulty, duration, available_slots, status, assigned_staff_id)
- **Booking** — Serves as the ledger log tracking trekker registration records (id, user_id, trek_id, booking_date, status)

Relationships:

- **One-to-Many** → User (Staff) → Trek (An approved staff member can be assigned to guide multiple treks)
- **One-to-Many** → User (Trekker) → Booking (A user can have multiple historical booking entries)
- **One-to-Many** → Trek → Booking (A trek can be booked by multiple trekkers)



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticates credentials and logs the User/Staff/Admin into a secure session
/register	POST	Registers a new Trekker account or submits a pending Staff profile entry
/admin/add_trek	POST	Programmatically appends a new trek itinerary slot route to the system
/admin/assign_staff/<id>	POST	Explicitly assigns an approved Staff guide to a specific Trek ID
/staff/update_trek/<id>	POST	Allows assigned staff to update remaining seats and toggle status (Open/Closed/Completed)
/user/profile	POST	Modifies the current logged-in trekker's contact particulars

7. Architecture and Features

Architecture Overview:

- **app.py** – Main standalone Flask application containing structural ORM models, routes, initialization logic, and programmatic seeding parameters.
- **/templates** – Contains raw Jinja2 HTML views ([login.html](#), [register.html](#), [admin_dashboard.html](#), [staff_dashboard.html](#), [user_dashboard.html](#)) completely styled with custom inline CSS blocks.

Implemented Features:

- **Programmatic DB Generation:** Recreates database files and seeds an active administrative superuser (`admin` / `admin123`) automatically on start.
- **No-JavaScript Dependency:** Complete data mutations, table rendering, filters, and dashboard actions run purely on server-side logic blocks.
- **Overbooking Shield:** Atomically checks seat limits and declines purchases if `available_slots <= 0`.
- **Access Control Guard:** Denies access and triggers explicit halts if blacklisted users or unapproved staff attempt dashboard entry.
- **History Log Tracking:** Preserves a user's full journey path timeline tracking booked, completed, or cancelled status variants.